reviews (e.g. Pitchfork[4]), social media posts (e.g. musigsuggestions subreddit[5]), or editorial content (e.g. rateyourmusic[6]), LLMs could identify and extract mentions of items or specific aspects associated with them. Moreover, LLMs could be directly integrated within conversational recommender systems and extract examples of liked and disliked music as well as preferred and disliked musical attributes [181] (e.g., "I like classical music but dislike repetitive vocals"). This process involves leveraging LLMs for NER, aligning an open vocabulary of short phrases with a fixed taxonomy, and high-level text reasoning to interpret expressions of dislike.

LLM as Summarizer or Captioner. LLMs are highly effective in generative tasks compared to any preceding models, particularly when the goal is to transform input data into new textual outputs with different structures or distributions from the source. In this role, LLMs are known to perform tasks such as: 1) *Summarization*—summarizing multiple sources of music item information into a concise, coherent summary or description; for instance, generating a short item caption from longer user descriptions focused on the interpretation of lyrics [108]. 2) *Item co-occurrence summarization*—analyzing patterns of music item co-occurrences and their metadata, then translating these insights into descriptive text that highlights relationships between items; an interesting application is generating smooth transitions or spoken segues between two tracks, similar to the commentary or explanations heard in radio shows [52]. 3) *Music captioning*–creating new content such as item captions or profiles, similar to the user profiles [37, 154].

While all these roles and examples focus on textual data as input, *Multi-modal Language Models* (MLMs) [34, 80], which can incorporate other signals such as an item's audio, can also be leveraged to augment music items with additional information such as tags, music characteristics, or more detailed descriptions.

2.3 Natural-Language Requests in Music Recommendation

We further focus on the setting where users express their music requests and preferences in natural language (NL). While communicating music preferences through language is arguably the most natural and socially grounded way of requesting music, resembling how people exchange recommendations among friends [181], it remains a highly challenging scenario for modern Music Streaming Platforms (MSPs). Thanks to their strong textual understanding capabilities, LLMs have made this one of the most affected, and perhaps most promising, areas of music recommendation.

Background. Given how natural it is for humans to articulate their needs in NL, MRSs with NL interfaces have emerged as a promising way for users to interact with the catalog [73]. While most MSPs provide a search bar that allows users to make textual queries to access the catalog directly, their functionalities remain limited. The search bar represents a high-control interaction mechanism, enabling users to access specific content, for instance, music from a particular artist or album [143]. Users also rely on search when they only have a "seed of an idea" about what they are looking for [86], such as "music to work out" or "music for studying." Despite the inherent limitations of traditional keyword-based search, users often adapt to its functioning and constraints over time.

The introduction of LLMs makes it possible to move beyond keyword matching or predefined frames in the case of traditional voice interaction [73], and to support more complex and expressive user requests. Several studies have explored integrating them into recommendation interfaces that connect users' natural-language requests to the catalog [29, 102, 111]. In this context, LLMs could draw on their extensive general "knowledge" of music to identify or describe relevant items. Trained on diverse internet text sources, they are able to capture and express musical attributes, such as tempo, rhythm, and timbre, through linguistic associations [16]. This opens the possibility of

[4]https://pitchfork.com
[5]https://www.reddit.com/r/musicsuggestions/
[6]https://rateyourmusic.com

bypassing traditional collaborative filtering pipelines and potentially mitigating issues such as popularity and exposure biases, which arise when models are trained solely on logged interaction data [21, 68, 100, 118, 158]. However, LLMs also introduce a new set of challenges that must be carefully addressed.

Music-specific challenges. Music makes for a particularly challenging recommendation scenario. It is a rich and multifaceted medium that spans cultures, countries, decades, and genres. Music catalogs typically contain millions of tracks, each of which can be described through metadata such as release year, genre, or country of origin. In addition, the audio signal itself carries a wealth of information that can be extracted—such as mood, tempo, or instrumentation [109, 115]. Music perception is both cultural [82] and dynamic [145, 146], varying across individuals and communities. Moreover, music consumption is often highly contextual: as music influences human cognition, emotion, and behavior, it can be deliberately employed for specific goals such as improving concentration, maintaining motivation, or expressing social identity [128]. Consequently, users often listen to music as a tool for emotional regulation, selecting tracks that match a particular context or activity [98].

These diverse dimensions make music an especially complex domain for natural-language requests. Users may ask for highly specific content (e.g., "Beatles songs from their early years") or for abstract, affective experiences (e.g., "party music with high energy"). The challenge lies in bridging these linguistic expressions with the underlying catalog representations—while also accounting for users' personal preferences, cultural background, and situational context.

Adding to this difficulty, new music tracks are continuously uploaded to MRS catalogs, creating persistent cold-start issues [46]. When these new tracks are absent from the LLM's training data, the model tends to favor older or more popular tracks, systematically overlooking newly released material. This is particularly problematic since many users rely on recommender systems precisely to discover new music [4]. Ultimately, NL-based music recommendation requires systems that can interpret language not only semantically but also affectively and contextually, aligning textual intent with both the structure of the catalog and the lived experience of music consumption.

Key changes introduced by LLMs. LLMs enable more flexible and expressive user queries in music recommendation. Moreover, through prompt design, their behavior could be guided or constrained—for instance, to mitigate biases related to popularity, gender, or other social factors [30]. In the following, we detail the different ways LLMs have been employed as music recommenders. We focus on the scenario in which a **ranked list of musical tracks** is produced in response to a user's natural-language request. This setting closely aligns with traditional top-K recommendation and playlist-generation tasks [29]. We define the following task components:

- *Music catalog:* track identifiers, canonical names, artist/album relations, and editorial or community metadata. Catalogs often contain multiple tracks sharing the same canonical names (e.g., homonymous artists, different versions of the same song across collections) and exhibit noisy or heterogeneous metadata, often supplied by labels, producers, or users.
- *Resolver/knowledge base:* used for entity validation and disambiguation (e.g., MusicBrainz; the Music Ontology is often used for schema-level reasoning) [105, 122].
- *NL user request:* e.g., "play some obscure bolero guitar pieces," "recommend me melancholic violin cues from anime soundtracks," or "play Brazilian forró with accordion solos."
- *NL user profile:* long- and short-term music preferences expressed in natural language [144].
- *In-context examples:* few-shot seeds / examples given as a list of tracks, artists, or other musical entities.
- *Retrieved documents for RAG:* e.g., metadata snippets, tags, reviews, or new-release notices [85].

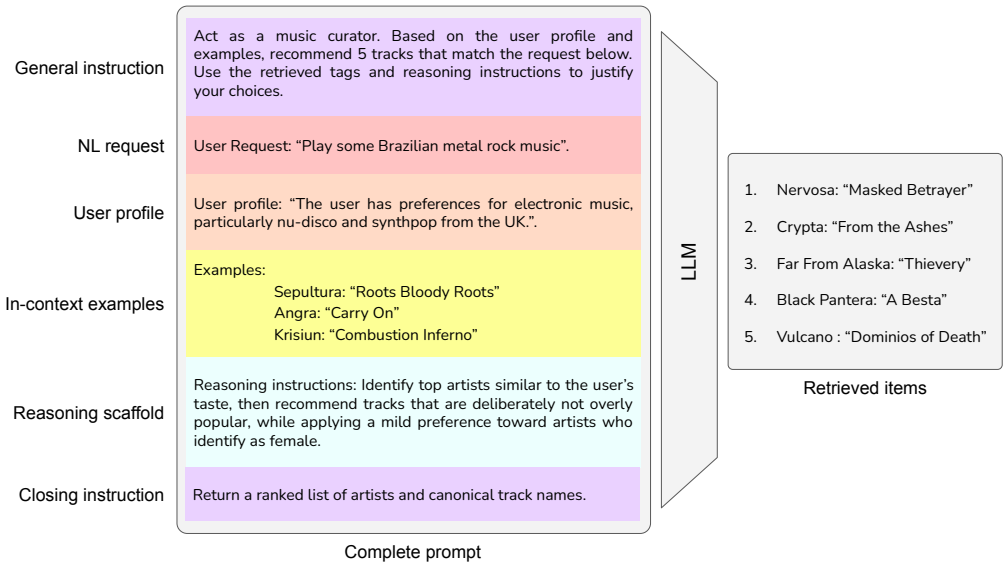

Fig. 3. Prompt example with the different task components highlighted.

- *Reasoning scaffold:* e.g., Chain-of-Thought or Tree-of-Thought prompting [170, 176].

As outlined by the list of task components, we assume that generation is grounded in a *resolver* or *knowledge base* (KB) and may be further structured through prompting scaffolds. Specifically, as the MIR community has long emphasized domain-specific challenges—such as *entity disambiguation* (artist, release, recording) [104, 125], *heterogeneous metadata* [12], and *long-tail discovery* [18, 46], we explicitly separate the catalog, which is often noisy, from knowledge bases, such as MusicBrainz, Discogs, or AcousticBrainz. The KB is then used to *validate the existence of and disambiguate entities* (e.g., homonyms, release variants, remasters), and to map generated item names back to the catalog in order to filter out hallucinations. Furthermore, a prompting operator composes these elements into a single prompt and a post-processor deduplicates, validates, and re-ranks the resulting list of tracks.

Figure 3 illustrates a complete prompt, split into panes. Different recommendation tasks correspond to combinations of these elements. Next we describe them in detail.

Zero-Shot Recommendation (user request-only, possibly conditioned by user profile | no in-context examples, retrieved documents, or reasoning scaffold). In this setting, following the example in Figure 3, only the user request (e.g., "Play some Brazilian metal rock music") is required, optionally accompanied by the NL user profile.

While classical recommender systems rely heavily on collected user feedback, zero-shot recommendation reframes the task as natural-language understanding, where the LLM maps open-vocabulary constraints to catalog items [13]. In the absence of a user profile, the model relies entirely on its linguistic priors. LLMs internalize broad cultural knowledge and can parse open-vocabulary descriptions, including compositional constraints, say, in terms of valence, arousal and dominance [167], key/mode, or content control (e.g., "avoid harsh vocals").

In profile-conditioned zero-shot, personalization is achieved by prepending a readable description of the user's taste and intent. The profile is interpretable, editable, and directly leverages behavior, unlike opaque latent vectors [173]. This "profile-as-text" approach enables controllable personalization without modifying model weights, allowing users or curators to make immediate corrections [173].

Zero-shot recommendation is particularly valuable in *exploratory* and *cold-start* scenarios because it operates without requiring any historical user interactions, instead treating the recommendation task as a language understanding and generation problem [13, 173]. Recent research demonstrates that zero-shot LLM-based recommenders excel at handling long-tail items and under-described genres that lack sufficient behavioral data [64, 65]. However, zero-shot approach carries inherent risks: without explicit grounding in retrieved documents or contextual information, these systems are susceptible to popularity bias and may generate plausible but incorrect recommendations (hallucinations) [74].

The fundamental challenge stems from the model's reliance on parametric knowledge—information encoded in its weights during pre-training. Recommendation quality therefore depends critically on two factors: accurate entity resolution that correctly maps user queries to items in the knowledge base (KB), and robust post-processing to validate outputs. When the model's parametric knowledge becomes stale relative to catalog updates, the zero-shot setting becomes particularly vulnerable to hallucinations, especially for niche or newly released items that were not present during training [74].

Despite these limitations, zero-shot approaches enable innovative user experience patterns. For example, systems can implement editable "taste cards" that allow users to modify their preferences and immediately see updated rankings without requiring model retraining [56, 110]. To harness this flexibility while mitigating risks, practitioners should implement stricter entity resolution mechanisms and post-validation procedures to prevent the system from generating confident but incorrect recommendations [74, 84].

In summary, zero-shot recommendation is a powerful tool for generating diverse suggestions and supporting exploratory discovery, provided that evaluation frameworks prioritize grounding accuracy and maintain balanced exposure across the item catalog.

Few-Shot In-Context Learning (ICL) (user request and in-context examples, possibly conditioned by user profile | no retrieved documents or reasoning scaffold). The zero-shot scenario is further extended by adding in-context examples—short listening histories or seed items—that capture user preferences. These examples can be sourced in two primary ways: from the target user's own interaction history (*personalized few-shot*) [30], enabling rapid adaptation to individual taste evolution, or from similar users' profiles in a collaborative setting (*collaborative few-shot*) [7, 178, 187]. Collaborative approaches are particularly valuable for cold-start scenarios or enriching sparse user profiles. Recent work explicitly designs systems to select similar user histories as in-context demonstrations, enabling LLMs to actively participate in choosing high-quality demonstration examples [187]. For instance, given the example in Figure 3, the items "Sepultura - Roots Bloody Roots", "Angra - Carry On", and "Krisiun - Combustion Inferno" serve as demonstrations to nudge the generation toward similar curation. Whether drawn from personal history or collaborative neighborhoods, the model leverages a few explicitly liked tracks or micro-histories to enable immediate personalization, well-suited for adapting to new or shifting tastes without re-training [7]. Few-shot ICL is particularly useful in session-based recommendation, where short-term intent can be efficiently reflected through a curated list of seed examples (see for example AdaptRec [187].)

However, selecting appropriate seed items requires careful consideration regardless of their source. Consumption logs are multi-intent and dominated by implicit signals (skips, repeats). If seeds are sampled naively from the chronological